Implementing Agentic RAG

Demo 1

© Brain4ce Education Solutions Pvt. Ltd. All rights reserved.

Demo: Create an AI-Powered Sales Report Analyzer with LlamaIndex

Overview

InsightPulse is an AI-powered tool for analyzing sales reports, aimed at delivering actionable insights derived from sales data. Utilizing LlamaIndex's VectorStoreIndex, Gemini embeddings, and a ReAct agent, it allows TechTrend Innovations to effectively examine extensive datasets, request sales information in natural language, and extract statistical metrics. The tool tackles issues like inefficiencies in manual analysis, inflexible querying options, and the demand for scalable insights.

Scenario

TechTrend Innovations, a firm engaged in electronics sales, oversees a substantial database of sales transactions. The sales team finds it difficult to gain insights because of the dataset's intricacy and the labor-intensive process of manual analysis. They require a solution capable of managing extensive datasets, facilitating natural language inquiries, and delivering prompt, scalable insights to aid strategic decision-making.

Problem Statement

TechTrend Innovations faces the following challenges:

- Large Dataset Complexity: Manually analyzing sales records is inefficient and error-prone.
- Flexible Query Requirements: Sales teams need adaptive, natural language queries to meet various analytical needs.
- Scalable Insights: The absence of an AI-powered tool hinders the ability to analyze and deliver immediate insights.

Solution

The solution, InsightPulse, addresses these challenges through the following steps:

- Index Sales Data: Use LlamaIndex's VectorStoreIndex with Gemini embeddings to index sales records for efficient retrieval.
- Build a ReAct Agent: Create a ReAct agent with multiple tools, including a query engine for data retrieval and a custom analytics tool for statistical computations.

© Brain4ce Education Solutions Pvt. Ltd. All rights reserved.

- Command-Line Interface: Implement an interactive command-line interface with query history, error handling, and extensibility for user interaction.

Dummy Data Creation for the Demo

```
import pandas as pd
import numpy as np
from datetime import datetime, timedelta

# Set a seed for reproducibility
np.random.seed(42)

# Define date range
start_date = datetime(2023, 1, 1)
end_date = datetime(2024, 12, 31)
num_days = (end_date - start_date).days
dates = [start_date + timedelta(days=i) for i in range(num_days)]

# Define possible values for each column
regions = ['North', 'South', 'East', 'West', 'Central']
products = ['Laptop', 'Keyboard', 'Mouse', 'Monitor', 'Webcam',
'Headphones']
categories = ['Electronics', 'Electronics', 'Electronics',
'Electronics', 'Electronics', 'Electronics']
quantities = np.random.randint(1, 5, size=len(dates))
unit_prices = np.random.uniform(50, 500, size=len(dates)).round(2)

# Create lists for each column
order_ids = [f'ORD{i+1:04d}' for i in range(len(dates))]
selected_dates = np.random.choice(dates, size=len(dates),
replace=False)
regions_data = np.random.choice(regions, size=len(dates))
products_data = np.random.choice(products, size=len(dates))
categories_data = [categories[products.index(prod)] for prod in
products_data]
total_sales = (quantities * unit_prices).round(2)

# Create a Pandas DataFrame
data = pd.DataFrame({
    'OrderID': order_ids,
    'Date': selected_dates,
    'Region': regions_data,
    'Product': products_data,
    'Category': categories_data,
    'Quantity': quantities,
```

© Brain4ce Education Solutions Pvt. Ltd. All rights reserved.

```
    'UnitPrice': unit_prices,
    'TotalSale': total_sales
})

# Save the DataFrame to a CSV file
data.to_csv('sales_data.csv', index=False)

print("sales_data.csv has been created successfully!")
```

- Reproducibility: np.random.seed(42) ensures consistent random data generation for reproducibility.
- Date Range: Generates dates between January 1, 2023, and December 31, 2024, to simulate a two-year sales period.
- Data Fields:
    - OrderID: Unique identifiers in the format ORD0001 to ORD0731 (based on the number of days).
    - Date: Randomly selected dates within the defined range.
    - Region: Randomly chosen from North, South, East, West, and Central.
    - Product: Randomly chosen from a list of electronics (Laptop, Keyboard, Mouse, Monitor, Webcam, Headphones).
    - Category: Set to Electronics for all products (aligned with the product list).
    - Quantity: Random integers between 1 and 4.
    - UnitPrice: Random floats between $50 and $500, rounded to two decimal places.
    - TotalSale: Computed as Quantity * UnitPrice, rounded to two decimal places.
- Output: The script saves the data to sales_data.csv, which is used by sales_analysis_agent.py for indexing and querying.

© Brain4ce Education Solutions Pvt. Ltd. All rights reserved.

Stepwise Solution

Project Structure:

The project is organized as follows:

- sales_analysis_agent.py: Main script containing the core logic for indexing, agent creation, and query handling.
- sales_data.csv: Dataset containing sales records (OrderID, Date, Region, Product, Category, Quantity, UnitPrice, TotalSale).
- requirements.txt: Lists dependencies (llama-index, pandas, google-generativeai, python-dotenv).
- .env: Stores environment variables (e.g., GOOGLE_API_KEY).
- index_storage/: Directory for persisting the VectorStoreIndex.

Step 1: Set Up Dependencies and Environment

The script begins by importing necessary libraries and setting up logging and environment variables.

- Purpose: Libraries like pandas handle data processing, llama_index provides indexing and querying capabilities, and dotenv manages environment variables.
- Logging: Configures logging to track the application's progress.
- Environment Variables: Loads the Google API key for Gemini models.

© Brain4ce Education Solutions Pvt. Ltd. All rights reserved.

```python
import os
import pandas as pd
from dotenv import load_dotenv
from llama_index.core import VectorStoreIndex, Document, StorageContext
from llama_index.llms.gemini import Gemini
from llama_index.embeddings.gemini import GeminiEmbedding
from llama_index.core.node_parser import SentenceSplitter
from llama_index.core.tools import QueryEngineTool, FunctionTool
from llama_index.core.agent import ReActAgent
import logging
from pathlib import Path

# Set up logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

# Load environment variables
load_dotenv()
GOOGLE_API_KEY = os.getenv("GOOGLE_API_KEY")
```

Step 2: Initialize LLM and Embeddings

The Gemini LLM and embedding models are initialized for natural language processing and vector embeddings.

Purpose: LLM is used for natural language understanding, while embed_model generates embeddings for indexing sales data.

```python
# Initialize Gemini LLM and embeddings
llm = Gemini(model_name="models/gemini-1.5-flash", api_key=GOOGLE_API_KEY)
embed_model = GeminiEmbedding(model_name="models/embedding-001", api_key=GOOGLE_API_KEY)
```

Step 3: Define a Custom Analytics Tool

A custom tool is created to compute statistical metrics (e.g., sum, average) on the sales data.

- Purpose: This tool allows the agent to perform calculations like total sales or average unit price, with optional filters (e.g., by region or year).

© Brain4ce Education Solutions Pvt. Ltd. All rights reserved.

- Implementation: Uses pandas to load and filter the CSV data, then computes the requested metric.

```python
def compute_analytics(metric: str, column: str, filter_condition: str = None) -> float:
    """Compute statistical metrics (e.g., sum, average) on sales data with optional filtering."""
    df = pd.read_csv("sales_data.csv")
    if filter_condition:
        df = df.query(filter_condition)
    if metric == "sum":
        return df[column].sum()
    elif metric == "average":
        return df[column].mean()
    return 0.0

analytics_tool = FunctionTool.from_defaults(
    fn=compute_analytics,
    name="analytics_tool",
    description="Computes statistical metrics (sum, average) on sales data with optional filters."
)
```

Step 4: Index Sales Data

The sales data is indexed using LlamaIndex's VectorStoreIndex, with persistence to disk for efficiency.

- Purpose: Converts the CSV data into a list of Document objects, then indexes them using VectorStoreIndex for efficient retrieval.
- Persistence: Checks if an index exists on disk; if not, creates a new one and saves it.
- SentenceSplitter: Splits documents into chunks to improve retrieval performance.

© Brain4ce Education Solutions Pvt. Ltd. All rights reserved.

```python
INDEX_STORAGE_DIR = "index_storage"

index_storage_path = Path(INDEX_STORAGE_DIR)
if index_storage_path.exists():
    logger.info("Loading existing index from disk...")
    storage_context = StorageContext.from_defaults(persist_dir=INDEX_STORAGE_DIR)
    index = VectorStoreIndex.load_from_disk(storage_context, embed_model=embed_model)
else:
    logger.info("Indexing sales data...")
    sales_df = pd.read_csv("sales_data.csv")

    documents = []
    for _, row in sales_df.iterrows():
        text = f"OrderID: {row['OrderID']}, Date: {row['Date']}, Region: {row['Region']}, " \
               f"Product: {row['Product']}, Category: {row['Category']}, Quantity: {row['Quantity']}, " \
               f"UnitPrice: {row['UnitPrice']}, TotalSale: {row['TotalSale']}"
        documents.append(Document(text=text))

    splitter = SentenceSplitter(chunk_size=512, chunk_overlap=50)
    index = VectorStoreIndex.from_documents(
        documents,
        embed_model=embed_model,
        transformations=[splitter]
    )
    index.storage_context.persist(persist_dir=INDEX_STORAGE_DIR)
    logger.info("Indexing complete and saved to disk.")
```

Step 5: Create a Query Engine and Tool

A query engine is created from the index and wrapped as a tool for the ReAct agent.

- Purpose: The query_engine retrieves relevant documents based on user queries, and the sales_tool integrates it into the agent's toolkit.
- Similarity_top_k: Limits the number of retrieved documents to the top 5 for efficiency.

```python
query_engine = index.as_query_engine(llm=llm, similarity_top_k=5)

sales_tool = QueryEngineTool.from_defaults(
    query_engine=query_engine,
    name="sales_data_tool",
    description="Provides insights from sales data including total sales, regional performance, and product analysis."
)
```

Step 6: Create a ReAct Agent

A ReAct agent is initialized with the sales and analytics tools.

- Purpose: The ReAct agent combines reasoning and action capabilities, using the tools to answer user queries.

© Brain4ce Education Solutions Pvt. Ltd. All rights reserved.

- Verbose Mode: Provides detailed logs of the agent's reasoning process.

```
agent = ReActAgent.from_tools([sales_tool, analytics_tool], llm=llm, verbose=True)
```

Step 7: Implement the Query Function

A function is defined to handle user queries and maintain query history.

Purpose: Processes the user's query using the agent and stores the query-response pair in the history.

```
def analyze_sales(query, query_history):
    response = agent.chat(query)
    query_history.append((query, str(response.response)))
    return response.response
```

Step 8: Create an Interactive Command-Line Interface

An interactive loop allows users to query the system, view history, or exit.

- Purpose: Provides a user-friendly interface for querying sales data.
- Features: Supports natural language queries, query history (last 5 queries), and error handling.
- Exit Condition: Users can type "exit" to quit the application.

© Brain4ce Education Solutions Pvt. Ltd. All rights reserved.

```python
if __name__ == "__main__":
    query_history = []
    print("Welcome to InsightPulse: Your AI-Powered Sales Report Analysis Tool!")
    print("Enter your query (e.g., 'What is the total sales for Laptops in South in 2024?')")
    print("Type 'history' to view recent queries, 'exit' to quit.")

    while True:
        user_query = input("\nYour query: ").strip()
        if user_query.lower() == "exit":
            print("Exiting InsightPulse. Goodbye!")
            break
        if user_query.lower() == "history":
            if query_history:
                print("\nRecent Queries:")
                for i, (q, r) in enumerate(query_history[-5:], 1):
                    print(f"{i}. Query: {q}\n   Response: {r[:100]}...")
            else:
                print("No query history yet.")
            continue
        if not user_query:
            print("Please enter a valid query.")
            continue

        print(f"\nProcessing query: {user_query}")
        try:
            response = analyze_sales(user_query, query_history)
            print(f"Response: {response}")
        except Exception as e:
            print(f"Error processing query: {e}")
```

Output of ReAct Agent

```
Your query: What is the total sales for Laptops in South in 2024?

Processing query: What is the total sales for Laptops in South in 2024?
> Running step 5f5aac8e-0e8f-4b6f-bf63-405a70151462. Step input: What is the total sales for Laptops in South in 2024?
Thought: The current language of the user is: English. I need to use a tool to get the total sales for Laptops in South in 2024.  I will use the `sales_data
tool` to get a general overview and then potentially use `analytics_tool` for specific calculations if needed.
Action: sales_data_tool
Action Input: {'input': 'Sales data for Laptops in South in 2024'}
Observation: Two laptop sales totaling 933.12 were recorded in the South region during 2024.

> Running step 35d93f5e-4912-4b11-8dcc-d2ff86a5ca37. Step input: None
Thought: I can answer without using any more tools. I'll use the user's language to answer
Answer: The total sales for Laptops in the South region in 2024 were $933.12.
Response: The total sales for Laptops in the South region in 2024 were $933.12.

Your query: history

Recent Queries:
1. Query: What is the total sales for Laptops in South in 2024?
   Response: The total sales for Laptops in the South region in 2024 were $933.12....

Your query: exit
Exiting InsightPulse. Goodbye!
```

© Brain4ce Education Solutions Pvt. Ltd. All rights reserved.

Map with Topics

The implementation of InsightPulse leverages concepts from the "Module Implementing Agentic RAG," which includes the following topics:

1. **Agentic RAG**

   Agentic Retrieval-Augmented Generation (RAG) is a framework that enhances traditional RAG by incorporating an agent capable of reasoning and acting on retrieved information. In InsightPulse, the ReAct agent exemplifies this by combining retrieval (via VectorStoreIndex) with actions (e.g., statistical computations).

2. **Agentic RAG vs. Traditional RAG**

   Traditional RAG retrieves and generates responses based on static retrieval, whereas Agentic RAG introduces dynamic decision-making. InsightPulse uses a ReAct agent to go beyond retrieval, enabling it to perform computations (e.g., via the analytics tool) and handle complex queries adaptively.

3. **Agentic RAG Architecture and Components**

   The architecture includes a retrieval system (VectorStoreIndex), an LLM (Gemini), embeddings (GeminiEmbedding), and an agent (ReActAgent). InsightPulse implements this by indexing sales data, creating tools for retrieval and analytics, and using the ReAct agent to orchestrate query handling.

4. **Understanding Adaptive RAG**

   Adaptive RAG adjusts its retrieval and reasoning based on query complexity. In InsightPulse, the ReAct agent adapts by choosing between the sales data tool (for retrieval) and the analytics tool (for computation) based on the query's requirements.

5. **Variants of Agentic RAG**

   Variants include ReAct, Reflexion, and others. InsightPulse specifically uses the ReAct variant, which combines reasoning and acting, as seen in the agent's ability to retrieve data and perform statistical analysis in a single workflow.

© Brain4ce Education Solutions Pvt. Ltd. All rights reserved.

6. **Agentic RAG with LlamaIndex**

   LlamaIndex is used to build the retrieval system in InsightPulse. The VectorStoreIndex indexes sales data, and the ReActAgent integrates with LlamaIndex tools (e.g., QueryEngineTool) to handle retrieval and reasoning, as shown in the indexing and query engine setup.

© Brain4ce Education Solutions Pvt. Ltd. All rights reserved.